Lab Guards: control de acceso y protección de rutas

App `auth-guard-app` · Ionic 9 + Angular 22 standalone · functional guards + signals

Jorge González Gómez · 2º DAM · Desarrollo de Interfaces (0488) · Universidad Nebrija · versión 2, 26-10-2026

Código: github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2/tree/main/08-Lab-Guards-Auth

Práctica completa, con la base del laboratorio y las implementaciones propuestas (apartado 4): dos roles y una factoría `roleGuard(allowedRole)`. Todo el código está comentado en GitHub.

App general de repaso. Para entender bien los conceptos de las prácticas 04–08 sin rehacerlas una a una, estoy haciendo una sola app de gestión con roles que junta lo importante de cada una (Supabase, guards, formularios reactivos y estado con signals). Está en el mismo repositorio: rama `app-gestion-roles`, carpeta `09-App-Gestion-Roles`.

1. Qué he hecho

App Ionic con login, un panel de administración protegido y un panel de operador. La sesión vive en un signal del `AuthService` y se guarda en `sessionStorage` (clave `DAM2_AUTH_SESSION`). Así sobrevive a un F5 o a escribir la URL a mano. Las rutas se protegen con guardas funcionales (`CanActivateFn`) que devuelven un `UrlTree` para redirigir.

Pieza	Archivo	Qué hace
Servicio	`core/services/auth.service.ts`	Usuarios `admin/123` (rol `admin`) y `operador/123` (rol `user`); login, logout y lectura del storage al arrancar
Guarda	`core/guards/auth.guard.ts`	¿Hay sesión? Si no, `UrlTree` a `/login?returnUrl=…`
Factoría	`core/guards/role.guard.ts`	`roleGuard('admin')`: si el rol no coincide, lo manda a su panel (`/user-dashboard?denegado=/admin`)
Rutas	`app.routes.ts`	Lazy loading; `/admin` con `[authGuard, roleGuard('admin')]`, `/user-dashboard` con `[authGuard]`
Vistas	`pages/login`, `pages/admin`, `pages/user-dashboard`	Formulario con `ngModel`; paneles que enseñan la sesión y el JSON guardado

2. Lo más interesante del código

La factoría de guardas es una función que recibe el rol y devuelve la guarda ya configurada, como un método que devuelve una lambda en Java:

```
export function roleGuard(allowedRole: string): CanActivateFn {
  return (route, state) => {
    const auth = inject(AuthService); const router = inject(Router);
    if (!auth.isAuthenticated()) return router.createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
    if (auth.role() === allowedRole) return true;
    // rol incorrecto: a SU panel, avisando de qué ruta se le ha denegado
    return router.createUrlTree([auth.rutaInicio(auth.role())], { queryParams: { denegado: state.url } });
  };
}

{ path: 'admin', canActivate: [authGuard, roleGuard('admin')],   // se ejecutan en orden
  loadComponent: () => import('./pages/admin/admin.page').then(m => m.AdminPage) }
```

Persistencia: el signal se inicializa con `signal(this.readSessionFromStorage())`. Por eso, al recargar, la sesión ya existe antes de que el router ejecute la guarda. Devolver un `UrlTree` en vez de `false` hace que la redirección sea inmediata y que `AdminPage` nunca llegue a cargarse; ni siquiera se descarga su chunk.

3. Pruebas de verificación (apartado 3)

Prueba	URL escrita	URL final	sessionStorage	
Entrar a la raíz	`/`	`/login`	vacío	OK
Login admin/123	—	`/admin`	`{"username":"admin","role":"admin","isAuthenticated":true}`	OK
Recargar (F5)	`/admin`	`/admin`	se mantiene	OK
Cerrar sesión	—	`/login`	clave eliminada	OK
Ataque por URL sin sesión	`/admin`	`/login?returnUrl=%2Fadmin`	vacío	OK
Login operador/123	—	`/user-dashboard`	`{"username":"operador","role":"user",…}`	OK
Rol user escribe /admin	`/admin`	`/user-dashboard?denegado=%2Fadmin`	se mantiene	OK

La clave también se ve en DevTools (F12 → Application → Session Storage → `http://localhost:8100`), y los paneles de admin y operador la muestran en pantalla.

4. Ejecutar

En `08-Lab-Guards-Auth/auth-guard-app`: `npm install` y `ionic serve` → `http://localhost:8100`.